OXFORDABLE CAREERS

WEEK 2 OF 3 · PARTICIPANT WORKBOOK v2.1

Design & Build Part 1

Data, Automation and Code

Name: _____

Cohort: _____

Date: _____

Salesforce Developer Introductory Programme · Labs 2, 3 and 4 · **Version 2.1 — Extension Pack update + AI Build Prompts appendix**

Read the solution design (Sections 1–2) before the session. We build from it, we do not re-explain it.

Welcome to Week 2

Last week you learnt what the job is. This week you do it: a designed data model, working automation, and your first Apex trigger — written by you, understood by you.

> **WHAT CHANGED IN VERSION 2.0 OF THIS WORKBOOK**
>
> The features listed as *out of scope* in Section 1 are no longer a dead end. After Week 3, the same project continues into the **CareerLaunch Extension Pack** — payments, invoicing, certificates, multi-currency/multi-language, error logging and a student portal — documented in the **Complete Build Guide** and the three Extension Workbooks (Parts A–C). Nothing you build today is throwaway: the Enrolment trigger you write in Lab 4 is the exact one the Extension Pack later extends by a single line.

Read before the session

Sections 1 and 2 of this workbook are the **requirements and solution design**. On a real project this is the document a developer receives before writing anything. Please read it before we meet — we will walk through it in the first 25 minutes rather than reading it aloud from cold.

Before you arrive — checklist

- ☐ My Developer Edition org opens and still contains the **Course** object and both Course records from Lab 1.
- ☐ VS Code is installed with the **Salesforce Extension Pack**.
- ☐ Running `sf --version` in a terminal returns a version number.
- ☐ Git is installed and I have a GitHub account.
- ☐ I have completed the Trailhead **Platform Basics** and **Data Modeling** modules.
- ☐ I have read Sections 1–2 of this workbook.

> **IF ANYTHING ABOVE IS NOT TICKED**
>
> Message the group **before** the session, not during it. Lab 2 is the foundation for everything in Weeks 2 and 3 — nobody is left behind on the data model, but we can only fix a broken install in advance.

What today covers

TIME	BLOCK	WHAT YOU WILL DO
0:00–0:10	Recap & homework check	Org health check, blockers
0:10–0:35	Solution design walkthrough	Architecture, the data model, why each relationship was chosen
0:35–1:15	**Lab 2 — Build the data model**	Cohort, Enrolment, relationships, roll-up, formulas, validation rules
1:15–1:25	Break	—
1:25–2:00	**Lab 3 — Declarative automation**	A record-triggered Flow and a screen Flow
2:00–2:50	**Lab 4 — Apex**	Trigger, handler, bulkification, Queueable, invocable Apex
2:50–3:00	Wrap-up & Homework 2	What breaks without tests — which sets up Week 3

BY THE END OF TODAY YOU WILL BE ABLE TO SAY

“I designed and built a three-object data model with the right relationships. I automated a business process with Flow. I wrote a bulkified Apex trigger with a handler class that enforces a capacity rule and blocks duplicates — and I can explain why each of those was the correct tool.”

1 — The Requirements Backlog

Here is the official backlog for the CareerLaunch project. Compare it with the stories you wrote in pairs last week. Each story has an ID used throughout this workbook, in the code comments, and in the Week 3 acceptance test script.

Scope — version 1 (Weeks 2–3)

IN SCOPE — WHAT WE BUILD NOW	EXTENSION SCOPE — DEFERRED, THEN DELIVERED
• Course, Cohort and Enrolment data model, with Contact as the Student • Seat capacity tracking and automatic prevention of overbooking • Automated welcome task and confirmation email • Automatic cohort status change to Full • A “Quick Enrol” screen-flow wizard • A scheduled reminder three days before the cohort starts • An Enrolment Console built from two custom LWCs • An Agentforce agent answering availability and status questions • Apex unit tests at 75% coverage or better • A permission set and a CLI deployment to a second org	• Payment gateway integration and invoicing • An Experience Cloud student portal • Certificate PDF generation • Multi-currency and multi-language • Org-wide error logging • Real external system integration — stubbed, with the pattern explained **New in v2.0:** every one of these is now built after Week 3 as the **CareerLaunch Extension Pack** — see the *Complete Build Guide* (Chapters 7–12) and Extension Workbooks Parts A–C.

> **SCOPE CONTROL IS STILL THE LESSON**
>
> Being able to explain *why* something is out of a two-week version 1 is itself part of the job. The client got a working v1 on time **because** the right-hand column waited. Deferred is not deleted — it is sequenced.

User stories

ID	USER STORY	ACCEPTANCE CRITERIA	PRIORITY
US-01	As an admissions officer, I want to record courses and the cohorts that run them, so that our catalogue lives in one place.	Course and Cohort objects exist; a Cohort must belong to a Course; a Cohort has start date, delivery mode, capacity and status.	Must
US-02	As an admissions officer, I want to enrol a student onto a cohort, so that attendance and payment can be tracked.	An Enrolment links one Contact to one Cohort; enrolment date defaults to today; status defaults to Applied.	Must
US-03	As an operations manager, I want to see how many seats remain on a cohort, so that I know what we can still sell.	Cohort shows Enrolled Count (excluding cancelled) and Seats Remaining = Capacity – Enrolled Count, both without manual entry.	Must

US-04	As an operations manager, I want the system to stop us overselling a cohort, so that we never promise a seat we do not have.	Saving an enrolment that would exceed capacity fails with a clear error; the rule holds when 200 records are loaded at once.	Must
US-05	As an admissions officer, I want a student to be prevented from being enrolled twice on the same cohort, so that our numbers are accurate.	A second enrolment for the same Contact and Cohort is rejected.	Must
US-06	As a student, I want a welcome message and next steps when my place is confirmed, so that I know what to do.	When Status changes to Confirmed: an email is sent to the student and a follow-up Task is created for the cohort owner.	Must
US-07	As an operations manager, I want a cohort marked Full automatically when the last seat is taken, so that it stops appearing as available.	When Seats Remaining reaches 0, Cohort Status becomes Full without manual intervention.	Must
US-08	As an admissions officer, I want a guided wizard to enrol a student in a few clicks, so that I do not have to create records manually.	A screen flow collects course → cohort → student and creates the enrolment, showing an error if the cohort is full.	Should
US-09	As a student, I want a reminder shortly before my cohort starts, so that I do not miss the first session.	A scheduled job creates a reminder three days before Start Date, for confirmed enrolments only.	Should
US-13	As an operations manager, I want each Contact to show how many active enrolments they hold, so that I can spot repeat students.	Contact shows a Total Enrolments count, updated automatically after enrolment changes.	Could

US-10 to US-12 (the Enrolment Console and the AI agent) and US-14 (testing and deployment) are delivered in Session 3 and appear in your Week 3 workbook. The Extension Pack continues the numbering with invoicing, payments, certificates, error logging and the portal.

Non-functional requirements

ID	REQUIREMENT
NFR-01	All Apex must be bulk-safe: correct behaviour for 1 record and for 200 records in a single transaction.
NFR-02	No SOQL queries or DML statements inside loops.
NFR-03	Apex code coverage of 75% or more (our target is 90%), with meaningful assertions — not coverage padding.
NFR-04	Errors surfaced to users must be human-readable, never a raw stack trace.
NFR-05	Access granted by permission set, never by editing the System Administrator profile.
NFR-06	All metadata must be retrievable and deployable via Salesforce CLI.

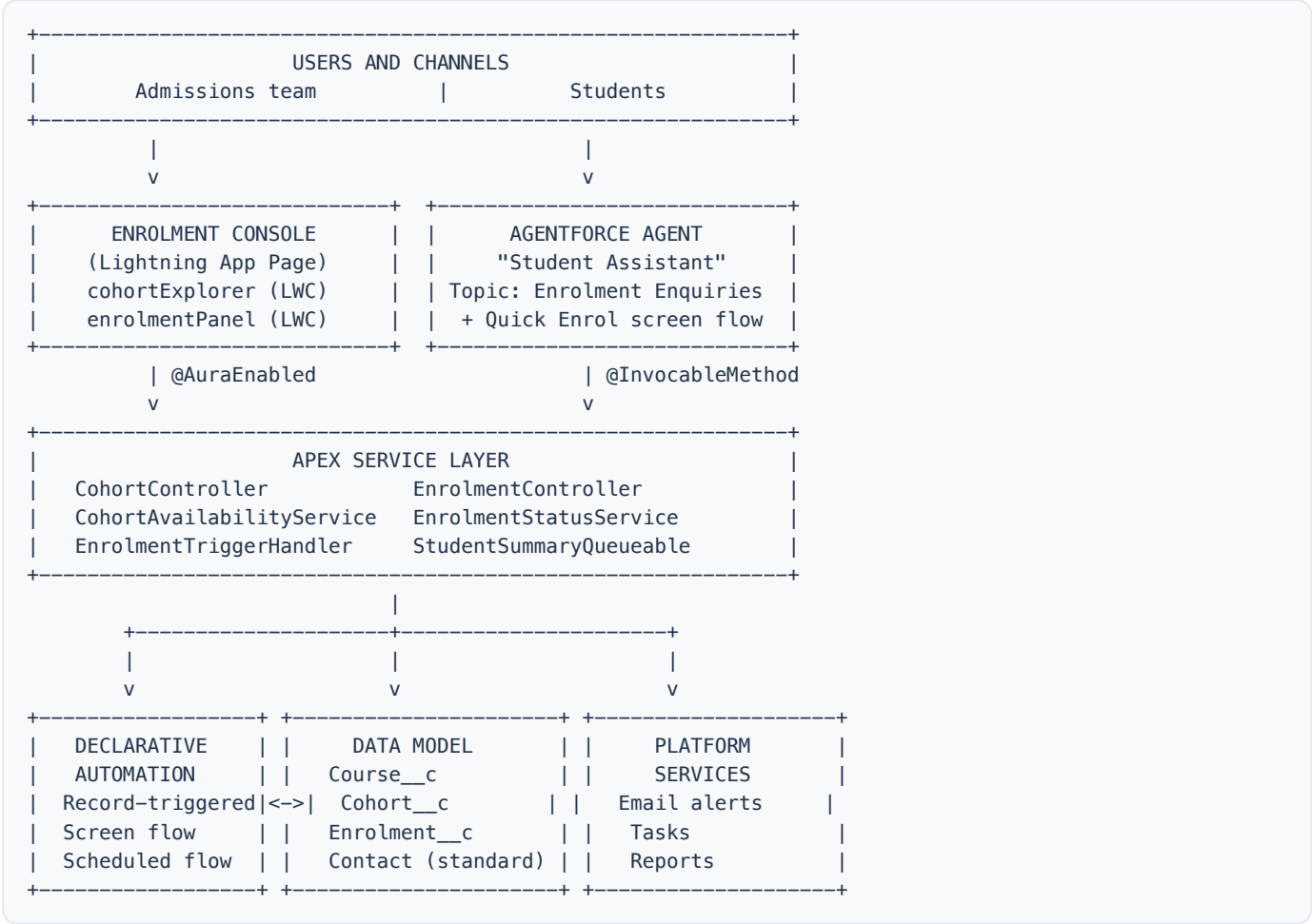
Assumptions

- The Student is a standard **Contact**. We do not build a custom Student object — reusing standard objects is good practice.
- One student holds at most one active enrolment per cohort.
- Payment is recorded as an amount only in **version 1**; the full gateway integration, invoicing and a Pay-now portal arrive in the Extension Pack (Build Guide Chapters 8–10 and 12).
- Email uses standard Salesforce email; deliverability is set to *All Email* in the Developer org.

2 — Solution Design

This is the design document you would hand to a client. We design first and build second — that is the habit this programme is trying to install.

Solution overview



THE LAYERING PRINCIPLE

The Lightning Web Component and the AI agent are only **channels**. Both call the same Apex service layer, which sits over the same data model, which is protected by the same automation. **Build the core once; expose it many times.** This one idea is worth more in an interview than any single piece of syntax — and it is why the Extension Pack can later add a whole student portal without touching this core.

2.1 The data model

RELATIONSHIP	TYPE	WHY
Cohort → Course	**Lookup**	A cohort should survive independently. We do not want cohorts deleted with a course, and we do not need a roll-up here.
Enrolment → Cohort	**Master-Detail**	We *do* want a roll-up summary of enrolments on the cohort, and an enrolment is meaningless without its cohort. Cascade delete is desirable.

Enrolment → Contact	**Lookup**	A contact may exist with no enrolments and must not be deleted when one is.

BE READY TO ANSWER THIS OUT LOUD

"Why master-detail here and lookup there?" — the single most examined design decision in PD1, asked in almost every junior interview. Have your answer before the session starts.

Object: Course__c — you built this in Lab 1

FIELD LABEL	API NAME	TYPE	NOTES
Course Name	`Name`	Text	Standard name field
Course Level	`Course_Level__c`	Picklist	Introductory, Advanced
Duration (Weeks)	`Duration_Weeks__c`	Number(2,0)	
Fee	`Fee__c`	Currency(10,2)	
Active	`Active__c`	Checkbox	Default: true
Description	`Description__c`	Long Text Area(1000)	

Object: Cohort__c — built in Lab 2

FIELD LABEL	API NAME	TYPE	NOTES
Cohort Number	`Name`	Auto Number	Format `CO-{0000}`
Course	`Course__c`	Lookup(Course__c)	Required
Start Date	`Start_Date__c`	Date	Required
End Date	`End_Date__c`	Date	
Delivery Mode	`Delivery_Mode__c`	Picklist	Online, In-Person, Hybrid
Capacity	`Capacity__c`	Number(3,0)	Required, default 20
Status	`Status__c`	Picklist	Planned, Open, Full, Running, Completed, Cancelled
Enrolled Count	`Enrolled_Count__c`	Roll-Up Summary	COUNT of Enrolment__c where Status ≠ Cancelled
Seats Remaining	`Seats_Remaining__c`	Formula (Number)	`Capacity__c - Enrolled_Count__c`
Cohort Label	`Cohort_Label__c`	Formula (Text)	`Course__r.Name & " - " & TEXT(Start_Date__c)`

Validation rule — `End_Date_After_Start_Date`

```
AND(
  NOT(ISBLANK(End_Date__c)),
  End_Date__c < Start_Date__c
)
```

Error message: *End Date cannot be earlier than Start Date.*

Object: Enrolment__c — built in Lab 2

FIELD LABEL	API NAME	TYPE	NOTES
Enrolment Number	`Name`	Auto Number	Format `ENR-{00000}`
Cohort	`Cohort__c`	Master-Detail(Cohort__c)	Required
Student	`Student__c`	Lookup(Contact)	Required
Status	`Status__c`	Picklist	Applied, Confirmed, Waitlisted, Cancelled, Completed. Default: Applied
Enrolment Date	`Enrolment_Date__c`	Date	Defaulted by Apex to today
Amount Paid	`Amount_Paid__c`	Currency(10,2)	Default 0 · the Extension Pack later writes this on payment
Enrolment Key	`Enrolment_Key__c`	Text(64) — External ID, Unique	Set by Apex to CohortId-StudentId. This is how we enforce US-05.
Attendance %	`Attendance_Percent__c`	Percent(3,0)	
Certificate Issued	`Certificate_Issued__c`	Checkbox	Ticked automatically by the Extension Pack's certificate job

Validation rule — `Payment_Not_Above_Fee`

```
Amount_Paid__c > Cohort__r.Course__r.Fee__c
```

Error message: *Amount Paid cannot exceed the course fee.*

> **NOTICE WHAT THAT RULE DOES**
>
> It reaches across **two relationships** in a single formula: Enrolment → Cohort → Course. This is a cross-object formula; Salesforce allows up to five levels.

Standard object: Contact (the Student)

FIELD LABEL	API NAME	TYPE	NOTES
Total Enrolments	`Total_Enrolments__c`	Number(3,0)	Maintained by the Queueable Apex job (US-13)

2.2 Automation design — which tool, and why

This is the most valuable table in the programme. You must be able to justify every choice in it — in the exam, and in an interview.

#	REQUIREMENT	TOOL CHOSEN	WHY NOT THE ALTERNATIVE
A1	Seat count on a cohort (US-03)	Roll-up summary + formula field	No automation needed at all. Always ask “can a field do this?” first.
A2	Welcome email and follow-up task on confirmation (US-06)	Record-triggered Flow (after save)	Declarative, no test class required, easily changed by an admin. Apex would be over-engineering.

A3	Set Cohort to Full when seats hit zero (US-07)	Record-triggered Flow (after save) updating the parent	Same reasoning as A2. Also demonstrates updating a related record from Flow.
A4	Prevent overbooking (US-04)	**Apex trigger** (before insert/update)	A validation rule cannot count sibling records. Flow cannot reliably block a bulk load with a clean per-record error. This needs cross-record logic and `addError()`.
A5	Prevent duplicate enrolment (US-05)	**Apex trigger** populating a unique external ID field	Elegant, database-enforced, bulk-safe and free at runtime. A great interview answer.
A6	Guided enrolment wizard (US-08)	Screen Flow calling invocable Apex	Screens are a Flow strength; the seat-availability lookup is reused from Apex.
A7	Pre-start reminder (US-09)	Scheduled-triggered Flow	Runs daily on a filtered set. A Schedulable Apex class would be needless code.
A10	Keep the Contact enrolment count fresh (US-13)	Queueable Apex called from the trigger	Aggregation is best done after the transaction commits. Also demonstrates asynchronous Apex and why we use it.

THE RULE OF THUMB TO MEMORISE

Field → Validation Rule → Flow → Apex. Move down the list only when the level above genuinely cannot do the job. Reaching for Apex first is the most common junior mistake. (The Extension Pack obeys the same rule: invoicing is Apex only because it needs idempotent cross-record logic; the certificate *trigger* is a Flow because a Flow is enough.)

Lab 2 — Build the Data Model

LAB 2 · 40 MINUTES · HANDS-ON

Build the Data Model

Create Cohort and Enrolment, wire up the relationships, add the calculated fields and validation rules, and load sample data.

NOBODY MOVES ON UNTIL THIS LAB IS FINISHED

Everything in the rest of the programme depends on this data model being correct. If you fall behind, say so — we will wait. Two spellings matter enormously: it is `Enrolment__c` with one **l** (British spelling) throughout, and field API names must match this workbook exactly or the Apex in Lab 4 will not compile.

1. **Create the Cohort object.** Setup → Object Manager → Create ▾ → Custom Object. Label/Plural `Cohort` / `Cohorts` · Record Name `Cohort Number` · Auto Number · format `CO-{0000}` · starting number 1 · tick Allow Reports, Allow Activities, **Launch New Custom Tab Wizard**.

2. **Add the Cohort fields** (Fields & Relationships → New). Leave *Enrolled Count, Seats Remaining, Cohort Label* until step 5 — they cannot be built yet.

FIELD	TYPE	SETTINGS
Course	Lookup Relationship	Related To: **Course**; tick Required
Start Date	Date	Tick Required
End Date	Date	—
Delivery Mode	Picklist	Online, In-Person, Hybrid
Capacity	Number	Length 3, Decimals 0, Default `20`, tick Required
Status	Picklist	Planned, Open, Full, Running, Completed, Cancelled

3. **Create the Enrolment object.** Same route. Label/Plural `Enrolment` / `Enrolments` · Record Name `Enrolment Number` · Auto Number `ENR-{00000}` · tick Allow Reports, Allow Activities, tab wizard.

4. **Add the Enrolment fields — in this order.** The master-detail must exist before the roll-up can be built.

FIELD	TYPE	SETTINGS
Cohort	Master-Detail Relationship	Related To: **Cohort**. Accept the sharing defaults.
Student	Lookup Relationship	Related To: **Contact**; tick Required
Status	Picklist	Applied, Confirmed, Waitlisted, Cancelled, Completed. Default **Applied**.
Enrolment Date	Date	—
Amount Paid	Currency	Length 10, Decimals 2, Default `0`
Enrolment Key	Text	Length 64; tick **External ID** and **Unique**; case-sensitive unticked
Attendance %	Percent	Length 3, Decimals 0
Certificate Issued	Checkbox	Default: Unchecked

> **PAUSE HERE — WHAT JUST HAPPENED**
> Creating that master-detail changed four things at once: the Cohort now **owns** the Enrolment; deleting a Cohort will **cascade delete** its Enrolments; **roll-up summaries** become possible on Cohort; and who can see an Enrolment is now **controlled by the parent** Cohort.

5. **Back on Cohort — add the three calculated fields.**

FIELD	TYPE	SETTINGS
Enrolled Count	Roll-Up Summary	Summarised object **Enrolments** · COUNT · filter: Status *not equal to* Cancelled
Seats Remaining	Formula → Number, 0 dp	`Capacity__c - Enrolled_Count__c`
Cohort Label	Formula → Text	`Course__r.Name & " - " & TEXT(Start_Date__c)`

6. **Add the Contact field.** Object Manager → Contact → Fields & Relationships → New → Number, Length 3, Decimals 0, Label `Total Enrolments`.

7. **Add the two validation rules** — copy the formulas and messages from Section 2.1 exactly. On Cohort: `End_Date_After_Start_Date`. On Enrolment: `Payment_Not_Above_Fee`.

8. **Tidy the page layouts.** Cohort layout: drag Enrolled Count, Seats Remaining, Cohort Label on; confirm the Enrolments related list. Confirm the Cohorts and Enrolments tabs appear in the App Launcher.

9. **Create the sample data.** Every later lab depends on this — do not skip.

OBJECT	RECORDS
Contact	Three students with real-looking **email addresses** — Week 3's agent searches on them.
Cohort	**CO-0001** — Introductory course · today + 14 · **Capacity 3** · Open **CO-0002** — Advanced course · today + 30 · Capacity 10 · Open

The capacity of 3 on CO-0001 is deliberate — it makes the overbooking test in Lab 4 fast.

CHECKPOINT

Create **one** Enrolment linking a Contact to CO-0001, then open CO-0001: **Enrolled Count = 1** and **Seats Remaining = 2** — without anybody typing a number. That is US-03 delivered, with no automation at all.

Lab 3 — Declarative Automation with Flow

LAB 3 · 35 MINUTES · HANDS-ON

Declarative Automation with Flow

A record-triggered Flow that welcomes a confirmed student and closes a full cohort, then a screen Flow that guides an admissions officer through enrolling somebody.

3a — Record-triggered Flow: `Enrolment_Confirmed_Actions` (US-06, US-07)

Setup → search **Flows** → New Flow → **Record-Triggered Flow**.

SETTING	VALUE
Object / Trigger	`Enrolment__c` · A record is created or updated
Entry conditions	`Status__c` Equals `Confirmed`
When to run	Only when a record is **updated to meet** the condition requirements
Optimise for	**Actions and Related Records** (after-save)

1. **Get Records** — label `Get_Cohort`. Object `Cohort__c`, condition Id Equals `{!$Record.Cohort__c}`. Store only the first record; fields `Seats_Remaining__c`, `Status__c`, `Start_Date__c`.
2. **Action → Send Email.** Recipient `{!$Record.Student__r.Email}` · Subject `Your place is confirmed – {!$Record.Cohort__r.Cohort_Label__c}` · Body: welcome text, start date, joining instructions.
3. **Create Records** — a Task. Subject `Send welcome pack to {!$Record.Student__r.Name}` · `WhatId = {!$Record.Id}` · `OwnerId = {!$Record.OwnerId}` · `ActivityDate = {!$Flow.CurrentDate}` · Priority High.
4. **Decision** — `Is_Cohort_Now_Full`. Outcome "Yes": `{!Get_Cohort.Seats_Remaining__c}` less than or equal to 0.
5. **Update Records** on Yes — update the Cohort from `Get_Cohort`, set `Status__c = Full`.
6. **Save** as `Enrolment Confirmed Actions`, then **Activate**. A saved flow that is not activated does nothing at all.

WHY AFTER-SAVE, NOT BEFORE-SAVE

We touch *other* records — an email, a Task, the parent Cohort. A before-save flow is faster but can only change fields on the triggering record. Classic PD1 question; you now own a concrete example.

TEST IT

Change an existing Enrolment's Status to **Confirmed**: a Task appears, the email arrives, and filling the final seat on CO-0001 flips the Cohort to **Full**.

3b — Screen Flow: `Quick_Enrol_Student` (US-08)

SCREEN / ELEMENT	CONTENT
Screen 1	Picklist choice set from active `Course__c` records — "Which course?"

Get Records	`Cohort__c` where Course = `{!selectedCourse}` AND Status = Open, sorted by Start Date, all records
Screen 2	Radio buttons from the cohort collection — display Cohort Label, seats remaining in help text
Screen 3	Lookup component for **Contact** — "Which student?"
Action	Apex Action → **Find Available Cohorts** (from `CohortAvailabilityService`) to re-check seats before writing
Decision / Create	Seats available? → Create `Enrolment__c` (Cohort, Student, Status = Confirmed)
Screen 4 (Yes / No)	Confirmation with the enrolment number / "Sorry — that cohort just filled up", with Back
Fault paths	Every Get and Create gets a **Fault** connector to an error screen showing `{!$Flow.FaultMessage}`. *(The Extension Pack later adds a "Log Flow Error" action on these same paths.)*

Save, activate, and surface it with a **Quick Enrol** button on the Cohort page or from the App Launcher.

> **SEQUENCING NOTE**
> The Apex Action needs the class you write in Lab 4. If we are short of time we build screens 1–3 and the Create element live; the Apex Action and fault handling become Homework 2, item 3.

3c — Scheduled Flow: `Cohort_Start_Reminder` (US-09) — demonstration, then homework

SETTING	VALUE
Trigger	Schedule-Triggered · daily at 07:00
Object	`Enrolment__c`
Entry conditions	`Status__c = Confirmed` AND `Cohort__r.Start_Date__c` equals today + 3 (build a formula resource)
Action	Send a reminder email and create a Task

Lab 4 — Apex: Trigger, Handler, Async and Invocable

LAB 4 · 50 MINUTES · HANDS-ON

Apex: Trigger, Handler, Async and Invocable

Write the code that enforces the seat capacity rule, blocks duplicate enrolments, updates the Contact asynchronously, and exposes a service that both Flow and an AI agent can call.

HOW THIS LAB RUNS

We **type the code together**, line by line. Please do not paste it in from Appendix A beforehand — watching code appear character by character is what demystifies it, and the errors you make while typing are the most useful part of the exercise. Appendix A is for catching up and revising.

4a — Apex in ten minutes

Developer Console → **Debug** → **Open Execute Anonymous Window**. We type and run this together:

```
// Variables, collections, loops, SOQL and DML in one page
String greeting = 'Hello Salesforce';
Integer capacity = 20;

List<String> levels = new List<String>{ 'Introductory', 'Advanced' };

Map<Id, Cohort__c> cohortsById = new Map<Id, Cohort__c>(
    [SELECT Id, Name, Capacity__c FROM Cohort__c WHERE Status__c = 'Open']
);

for (Cohort__c c : cohortsById.values()) {
    System.debug(greeting + ' – ' + c.Name + ' has capacity ' + c.Capacity__c);
}

Course__c newCourse = new Course__c(Name = 'Data Cloud Fundamentals',
                                    Course_Level__c = 'Introductory');
insert newCourse;
```

CONCEPT	WHAT TO TAKE AWAY
Strongly typed	Every variable declares its type. The compiler catches mistakes before your users do.
List, Set, Map	The three collections you will use constantly. A **Map** is a lookup table — keys to values — and the key to efficient Apex.
SOQL / DML	SOQL reads data (in square brackets); DML (`insert, update, delete, upsert`) writes it.
Governor limits	100 SOQL queries, 150 DML statements per transaction. This is *why* we bulkify.

4b — The trigger and the handler (US-04, US-05)

Two files: a **trigger** that does nothing but route (Appendix A.1), and a **handler class** that holds all the logic (Appendix A.2).

BEAT	THE POINT BEING MADE
One trigger per object	With more than one, the order they run in is unpredictable.

No logic in the trigger	Handler classes can be unit tested, reused and read. Triggers cannot be called from anywhere else.
`Trigger.new`, `Trigger.oldMap`	Context variables — the records as they will be, and as they were.
Collect IDs → one query → loop in memory	**The bulkification pattern.** Learn this shape; it is most of what separates working Apex from Apex that fails in production.
`addError()`	Blocks the individual record with a readable message, rather than failing the whole batch.
Unique External ID field	Duplicate prevention enforced by the database itself, at no runtime cost.

Methods we build inside `EnrolmentTriggerHandler`: `setDefaults` (US-02) · `buildUniqueKey` (US-05) · `preventOverbooking` (US-04, NFR-01) · `enqueueStudentSummary` (US-13).

4c — Asynchronous Apex: `StudentSummaryQueueable` (US-13)

Aggregating a Contact's enrolments is best done **after** the transaction commits — and an asynchronous job gets its own fresh set of governor limits. That is the whole argument for asynchronous Apex, in one sentence. (Appendix A.3.)

4d — Invocable Apex: `CohortAvailabilityService` (US-08, US-11)

The bridge class: **one method, three consumers** — the screen flow from Lab 3, the Agentforce agent in Week 3, and (with a wrapper) any external system. (Appendix A.4.)

```
@InvocableMethod(label='Find Available Cohorts' ...)
```

SAY THIS ONE OUT LOUD

That single annotation is what makes an AI agent able to use your code — arguably the most commercially valuable line in the whole project. In Week 3 you watch an agent call it; in the Extension Pack the same idea (invocable Apex) powers flow-driven certificates and error logging.

CHECKPOINT — TWO ACCEPTANCE CRITERIA PROVEN LIVE

☐ Enrol a fourth student onto CO-0001 (capacity 3): the save must fail with *your* message.
☐ Enrol an existing student onto the same cohort twice: that must fail too.
US-04 and US-05, demonstrably delivered.

Homework 2 — due before Session 3

Roughly 2 hours 30 minutes. Session 3 assumes the data model, the flows and the Apex from today are all working in your org.

#	TASK	ESTIMATE
1	Finish anything unfinished from Labs 2, 3 and 4. Use Appendix A to catch up.	30 min
2	Build the scheduled flow `Cohort_Start_Reminder` (Lab 3c)	20 min
3	Add the Apex Action and the fault paths to `Quick_Enrol_Student`	20 min
4	Trailhead: complete the **Apex Basics & Database** module	45 min
5	**Write one test method** for `EnrolmentTriggerHandler` — any method, however rough. Do not look up how first. Just try.	20 min
6	Push your work to GitHub: `sf project retrieve start`, then commit	15 min

> **ABOUT TASK 5**
> You have deliberately been asked to write a test before anybody has taught you testing. Struggling with it for twenty minutes is the point — you will arrive at Session 3 having already felt the problem, which is what makes the lesson land. Bring whatever you produce, working or not.

Troubleshooting — this week's greatest hits

PROBLEM	CAUSE	FIX
Roll-Up Summary is not offered as a field type	The child is related by Lookup, not Master-Detail	Recreate the relationship on `Enrolment__c` as Master-Detail
I cannot change a Lookup to Master-Detail	Some child records have a blank parent	Populate the parent on every child record first
“field integrity exception” on a formula	Referencing a relationship that does not exist	Check the relationship name: `Course__r.Name`, not `Course__c.Name`
My trigger does not fire	Inactive, or the event is missing from the signature	Check `(before insert, before update, ...)` and that it is Active
Flow saved but nothing happens	Flows do nothing until activated	Open the flow and click **Activate**
The welcome email never arrives	Deliverability is “System email only”	Setup → Deliverability → Access level: **All Email**
`Too many SOQL queries: 101`	A query inside a loop	Collect the IDs first, then query once
Compile error on a field name	`Enrollment__c` vs `Enrolment__c`	British spelling, one **l**, everywhere

Key terms from this session

TERM	MEANING

Lookup	A loose relationship; the child survives the parent.
Master-Detail	Tight parent-child: cascade delete, inherited sharing, roll-up summaries.
Roll-Up Summary	A master-record field that counts, sums, or takes min/max of its details.
Validation rule	A formula that blocks a save when it evaluates to true.
External ID	An outside-system identifier field; can be unique and used for upsert.
Flow	Declarative automation: record-triggered, screen, scheduled, autolaunched.
Apex / Trigger	Salesforce's Java-like server language / Apex that runs before or after saves.
Bulkification	Correct, efficient behaviour when processing many records at once.
Governor limit	Per-transaction caps that keep a multi-tenant platform fair.
SOQL / DML	How Apex reads data / writes data.
Queueable Apex	An asynchronous job with fresh limits, enqueued with `System.enqueueJob`.
Invocable method	`@InvocableMethod` Apex that Flow, Agentforce and REST can call.

SEE YOU NEXT WEEK — AND AFTER THAT

Session 3 — Build (Part 2), Test, Deploy & Demo: two Lightning Web Components, an Agentforce agent calling today's Apex, a test suite, a deployment and a demo. **Then the Extension Pack:** payments & invoicing, certificates, error logging, multi-currency/multi-language and the Experience Cloud student portal — the entire right-hand column of Section 1, built on the exact core you finished today. Reserve a place on the Advanced Course — WhatsApp +44 7448 717334 · +234 704 916 5925.

Appendix A — Apex Source Code

> **PLEASE READ THIS FIRST**
>
> This appendix is for **catching up and revising**, not for reading ahead. In Lab 4 you type this code with the instructor. If you paste it in beforehand you will get a working org and learn considerably less. If you fall behind during the lab, copy from here and rejoin the group. All classes use the British spelling `Enrolment__c` consistently. Copy exactly.

A.1 EnrolmentTrigger

```
trigger EnrolmentTrigger on Enrolment__c (before insert, before update, after insert, after update) {
    if (Trigger.isBefore) {
        if (Trigger.isInsert) {
            EnrolmentTriggerHandler.onBeforeInsert(Trigger.new);
        } else if (Trigger.isUpdate) {
            EnrolmentTriggerHandler.onBeforeUpdate(Trigger.new, Trigger.oldMap);
        }
    }
    if (Trigger.isAfter && (Trigger.isInsert || Trigger.isUpdate)) {
        EnrolmentTriggerHandler.onAfterChange(Trigger.new);
    }
}
```

The trigger contains routing only. All logic lives in a class that can be unit tested, reused and read.

A.2 EnrolmentTriggerHandler

```
public with sharing class EnrolmentTriggerHandler {

    @TestVisible
    private static final String ERR_FULL =
        'This cohort is full. Please choose another cohort or add the student to the waiting list.';

    // ---------- Entry points ----------
    public static void onBeforeInsert(List<Enrolment__c> newEnrolments) {
        setDefaults(newEnrolments);
        buildUniqueKey(newEnrolments);
        preventOverbooking(newEnrolments, null);
    }

    public static void onBeforeUpdate(List<Enrolment__c> newEnrolments,
                                      Map<Id, Enrolment__c> oldMap) {
        buildUniqueKey(newEnrolments);
        preventOverbooking(newEnrolments, oldMap);
    }

    public static void onAfterChange(List<Enrolment__c> newEnrolments) {
        enqueueStudentSummary(newEnrolments);
    }

    // ---------- US-02: sensible defaults ----------
    private static void setDefaults(List<Enrolment__c> newEnrolments) {
        for (Enrolment__c enr : newEnrolments) {
            if (enr.Enrolment_Date__c == null) {
                enr.Enrolment_Date__c = Date.today();
            }
            if (String.isBlank(enr.Status__c)) {
                enr.Status__c = 'Applied';
            }
            if (enr.Amount_Paid__c == null) {
```

```
            enr.Amount_Paid__c = 0;
        }
    }
}

// ---------- US-05: duplicate prevention via a unique External ID ----------
private static void buildUniqueKey(List<Enrolment__c> newEnrolments) {
    for (Enrolment__c enr : newEnrolments) {
        if (enr.Cohort__c != null && enr.Student__c != null) {
            enr.Enrolment_Key__c =
                String.valueOf(enr.Cohort__c).left(15) + '-' +
                String.valueOf(enr.Student__c).left(15);
        }
    }
}

// ---------- US-04: capacity enforcement, bulk safe ----------
private static void preventOverbooking(List<Enrolment__c> newEnrolments,
                                       Map<Id, Enrolment__c> oldMap) {

    // 1. Work out which enrolments newly consume a seat, and how many per cohort.
    Map<Id, Integer> incomingSeatsByCohort = new Map<Id, Integer>();

    for (Enrolment__c enr : newEnrolments) {
        if (enr.Cohort__c == null) {
            continue;
        }
        Boolean consumesSeat = enr.Status__c != 'Cancelled';
        Boolean isNewlyConsuming;

        if (oldMap == null) {
            isNewlyConsuming = consumesSeat;                        // insert
        } else {
            Enrolment__c older = oldMap.get(enr.Id);
            Boolean wasConsuming = older.Status__c != 'Cancelled'
                                && older.Cohort__c == enr.Cohort__c;
            isNewlyConsuming = consumesSeat && !wasConsuming;       // update
        }

        if (isNewlyConsuming) {
            Integer running = incomingSeatsByCohort.containsKey(enr.Cohort__c)
                ? incomingSeatsByCohort.get(enr.Cohort__c) : 0;
            incomingSeatsByCohort.put(enr.Cohort__c, running + 1);
        }
    }

    if (incomingSeatsByCohort.isEmpty()) {
        return;
    }

    // 2. ONE query for every cohort involved - never inside the loop.
    Map<Id, Cohort__c> cohortsById = new Map<Id, Cohort__c>([
        SELECT Id, Name, Capacity__c, Enrolled_Count__c
        FROM Cohort__c
        WHERE Id IN :incomingSeatsByCohort.keySet()
    ]);

    // 3. Allocate seats in order and error on the ones that do not fit.
    Map<Id, Integer> allocated = new Map<Id, Integer>();

    for (Enrolment__c enr : newEnrolments) {
        if (enr.Cohort__c == null || !incomingSeatsByCohort.containsKey(enr.Cohort__c)) {
            continue;
        }
        Cohort__c cohort = cohortsById.get(enr.Cohort__c);
        if (cohort == null) {
            continue;
```

```
            }

            Decimal capacity  = cohort.Capacity__c == null ? 0 : cohort.Capacity__c;
            Decimal committed = cohort.Enrolled_Count__c == null ? 0 : cohort.Enrolled_Count__c;
            Integer used      = allocated.containsKey(enr.Cohort__c)
                                ? allocated.get(enr.Cohort__c) : 0;

            if (committed + used + 1 > capacity) {
                enr.addError(ERR_FULL);
            } else {
                allocated.put(enr.Cohort__c, used + 1);
            }
        }
    }

    // ---------- US-13: asynchronous aggregation onto the Contact ----------
    private static void enqueueStudentSummary(List<Enrolment__c> newEnrolments) {
        Set<Id> studentIds = new Set<Id>();
        for (Enrolment__c enr : newEnrolments) {
            if (enr.Student__c != null) {
                studentIds.add(enr.Student__c);
            }
        }
        if (studentIds.isEmpty()) {
            return;
        }
        // Respect the asynchronous job limit for the transaction.
        if (Limits.getQueueableJobs() < Limits.getLimitQueueableJobs()) {
            System.enqueueJob(new StudentSummaryQueueable(studentIds));
        }
    }
}
```

VERSION 2.0 NOTE

In the Extension Pack, `onAfterChange` gains exactly **one more line** — a call to `InvoiceService.createForConfirmedEnrolments(newEnrolments)` that raises an invoice whenever an enrolment is confirmed. Do **not** add it this week (the class does not exist yet); see the Complete Build Guide, Chapter 10.

A.3 StudentSummaryQueueable

```
public with sharing class StudentSummaryQueueable implements Queueable {

    private final Set<Id> studentIds;

    public StudentSummaryQueueable(Set<Id> studentIds) {
        this.studentIds = studentIds;
    }

    public void execute(QueueableContext context) {
        Map<Id, Contact> contactsToUpdate = new Map<Id, Contact>();

        // Start everyone at zero so cancellations are reflected too.
        for (Id studentId : studentIds) {
            contactsToUpdate.put(studentId, new Contact(Id = studentId, Total_Enrolments__c = 0));
        }

        for (AggregateResult ar : [
            SELECT Student__c studentId, COUNT(Id) total
            FROM Enrolment__c
            WHERE Student__c IN :studentIds
              AND Status__c != 'Cancelled'
            GROUP BY Student__c
```

```
        ]) {
            Id studentId  = (Id) ar.get('studentId');
            Integer total = (Integer) ar.get('total');
            contactsToUpdate.put(studentId, new Contact(Id = studentId, Total_Enrolments__c = total));
        }

        if (!contactsToUpdate.isEmpty()) {
            try {
                update contactsToUpdate.values();
            } catch (DmlException e) {
                System.debug(LoggingLevel.ERROR, 'Student summary update failed: ' + e.getMessage());
            }
        }
    }
}
```

A.4 CohortAvailabilityService — invocable; used by Flow and Agentforce

```
public with sharing class CohortAvailabilityService {

    public class Request {
        @InvocableVariable(
            label='Course Name'
            description='All or part of the course name, for example "Advanced" or "Salesforce
Developer".'
            required=true)
        public String courseName;
    }

    public class Result {
        @InvocableVariable(
            label='Answer'
            description='A short, human-readable summary of the cohorts that still have seats.')
        public String answer;

        @InvocableVariable(
            label='Available Cohorts'
            description='The matching cohort records that still have seats available.')
        public List<Cohort__c> cohorts;

        @InvocableVariable(label='Seats Available' description='True if at least one seat was found.')
        public Boolean seatsAvailable;
    }

    @InvocableMethod(
        label='Find Available Cohorts'
        description='Returns open cohorts that still have seats for a given course name.'
        category='Oxfordable CareerLaunch')
    public static List<Result> findAvailableCohorts(List<Request> requests) {

        List<Result> results = new List<Result>();

        for (Request req : requests) {
            String searchTerm = '%' + (String.isBlank(req.courseName) ? '' : req.courseName.trim()) +
'%';

            List<Cohort__c> candidates = [
                SELECT Id, Name, Cohort_Label__c, Course__r.Name, Start_Date__c,
                       Delivery_Mode__c, Capacity__c, Enrolled_Count__c, Seats_Remaining__c
                FROM Cohort__c
                WHERE Status__c = 'Open'
                  AND Start_Date__c >= TODAY
                  AND Course__r.Name LIKE :searchTerm
                ORDER BY Start_Date__c ASC
                LIMIT 20
```

```
            ];

            List<Cohort__c> withSeats = new List<Cohort__c>();
            for (Cohort__c c : candidates) {
                if (c.Seats_Remaining__c != null && c.Seats_Remaining__c > 0) {
                    withSeats.add(c);
                }
            }

            Result res         = new Result();
            res.cohorts        = withSeats;
            res.seatsAvailable = !withSeats.isEmpty();
            res.answer         = buildAnswer(req.courseName, withSeats);
            results.add(res);
        }
        return results;
    }

    private static String buildAnswer(String courseName, List<Cohort__c> cohorts) {
        if (cohorts.isEmpty()) {
            return 'There are currently no cohorts with available seats matching "'
                 + courseName + '". Please ask about another course or a later date.';
        }
        List<String> lines = new List<String>();
        for (Cohort__c c : cohorts) {
            lines.add(c.Course__r.Name + ' starting ' + c.Start_Date__c.format()
                    + ' (' + c.Delivery_Mode__c + ') – '
                    + String.valueOf(c.Seats_Remaining__c.intValue()) + ' seat(s) remaining');
        }
        return 'Cohorts with availability: ' + String.join(lines, '; ') + '.';
    }
}
```

Appendix B — AI Build Prompts

Each prompt below, pasted into an AI coding assistant (Claude Code, Agentforce for Developers, or similar) running **inside your careerlaunch SFDX project with your org authorised**, rebuilds one layer of the application. The prompts encode the exact specification from this workbook — API names, picklist values, formulas — so the result matches what the labs build by hand.

READ THIS BEFORE USING ANY PROMPT

The labs come first. You learn by typing, breaking and fixing — not by generating. Use these prompts for three legitimate purposes only: **catching up** after the session if your org is broken, **verifying** your hand-built org against a correct build, or **rebuilding fast** in a fresh org (for example, before UAT practice). Whatever the AI produces: read the diff before deploying, deploy with the CLI yourself, and run the checkpoints from Labs 2–4 to prove the result. If you cannot explain a line the AI wrote, you are not finished with it.

SETUP FOR EVERY PROMPT

1. Open a terminal in your `careerlaunch` project folder (created in Week 1).
2. Confirm the org connection: `sf org list` shows your org as default.
3. Start your AI assistant in that folder, paste one prompt, review, deploy, run the matching lab checkpoint.
4. Run the prompts **in order B.1 → B.5** — later layers depend on earlier ones.

B.1 — Prompt: create all objects and fields

```
You are working in a Salesforce DX project connected to my Developer org. Create the metadata
(source format, under force-app/main/default) for the CareerLaunch data model, then deploy it.
Use these EXACT API names and settings - do not improvise names. British spelling: Enrolment, one "l".

1) Custom object Course__c (may already exist from Lab 1 - if so, verify and complete it):
   label Course/Courses, Name field = text "Course Name", enable reports and activities.
   Fields: Course_Level__c Picklist restricted (Introductory, Advanced);
   Duration_Weeks__c Number(2,0); Fee__c Currency(10,2); Active__c Checkbox default true;
   Description__c LongTextArea(1000).

2) Custom object Cohort__c: label Cohort/Cohorts, Name = AutoNumber "CO-{0000}", reports+activities.
   Fields: Course__c Lookup(Course__c) required; Start_Date__c Date required; End_Date__c Date;
   Delivery_Mode__c Picklist (Online, In-Person, Hybrid); Capacity__c Number(3,0) required default 20;
   Status__c Picklist (Planned, Open, Full, Running, Completed, Cancelled);
   Enrolled_Count__c Roll-Up Summary COUNT of Enrolment__c filtered Status__c != Cancelled;
   Seats_Remaining__c Formula Number(0dp) = Capacity__c - Enrolled_Count__c;
   Cohort_Label__c Formula Text = Course__r.Name & " - " & TEXT(Start_Date__c).
   Validation rule End_Date_After_Start_Date:
   AND(NOT(ISBLANK(End_Date__c)), End_Date__c < Start_Date__c)
   message "End Date cannot be earlier than Start Date."

3) Custom object Enrolment__c: label Enrolment/Enrolments, Name = AutoNumber "ENR-{00000}".
   Fields in this order: Cohort__c MasterDetail(Cohort__c); Student__c Lookup(Contact) required;
   Status__c Picklist restricted (Applied default, Confirmed, Waitlisted, Cancelled, Completed);
   Enrolment_Date__c Date; Amount_Paid__c Currency(10,2) default 0;
   Enrolment_Key__c Text(64) External ID + Unique (case-insensitive);
   Attendance_Percent__c Percent(3,0); Certificate_Issued__c Checkbox default false.
   Validation rule Payment_Not_Above_Fee:
   Amount_Paid__c > Cohort__r.Course__r.Fee__c
   message "Amount Paid cannot exceed the course fee."

4) On standard Contact: field Total_Enrolments__c Number(3,0), label "Total Enrolments".

5) Custom tabs for Course__c, Cohort__c and Enrolment__c (any motif).
```

```
Then: deploy with sf project deploy start, report any errors, and list every file you created.
Do NOT create any Apex, flows, LWCs or sample records in this step.
```

B.2 — Prompt: create all the Apex classes

```
In the same project (data model from prompt B.1 is deployed), create these Apex files exactly as
specified by the Oxfordable Week 2/3 workbooks, deploy them, then run all tests and show me the results.
API version 62.0, with sharing unless stated, British spelling Enrolment__c throughout.

1) trigger EnrolmentTrigger on Enrolment__c (before insert, before update, after insert, after update)
   - routing only, no logic: before insert -> EnrolmentTriggerHandler.onBeforeInsert(Trigger.new);
   before update -> onBeforeUpdate(Trigger.new, Trigger.oldMap); after insert/update ->
   onAfterChange(Trigger.new).

2) class EnrolmentTriggerHandler with: setDefaults (Enrolment_Date__c=today, Status__c='Applied',
   Amount_Paid__c=0 when blank); buildUniqueKey (Enrolment_Key__c = first 15 chars of Cohort__c
   + '-' + first 15 chars of Student__c); preventOverbooking (bulk-safe: count newly-consuming
   rows per cohort where Status != 'Cancelled', ONE query of Capacity__c/Enrolled_Count__c for the
   involved cohorts, allocate seats in memory, addError("This cohort is full. Please choose another
   cohort or add the student to the waiting list.") on rows that exceed capacity);
   enqueueStudentSummary (collect Student__c ids, enqueue StudentSummaryQueueable, respecting
   Limits.getQueueableJobs()).

3) class StudentSummaryQueueable implements Queueable: start every passed Contact at
   Total_Enrolments__c = 0, then one aggregate query COUNT(Id) grouped by Student__c where
   Status__c != 'Cancelled', update the Contacts, swallow DmlException with a debug log.

4) class CohortAvailabilityService: @InvocableMethod label 'Find Available Cohorts'
   category 'Oxfordable CareerLaunch'; Request has courseName; Result has answer, cohorts,
   seatsAvailable; query Open cohorts starting today or later whose Course name LIKE the term,
   keep those with Seats_Remaining__c > 0, build a human-readable answer string.

5) class EnrolmentStatusService: @InvocableMethod label 'Check Enrolment Status', Request has
   studentEmail; returns found + an answer summarising each enrolment (cohort label, start date,
   status), with a polite not-found message.

6) class CohortController: @AuraEnabled(cacheable=true) getActiveCourses() and
   getOpenCohorts(Id courseId) - Open, future cohorts, WITH SECURITY_ENFORCED, optional course filter.

7) class EnrolmentController: @AuraEnabled createEnrolment(Id cohortId, Id studentId, String status)
   - validate inputs, insert with Status default 'Confirmed', catch DmlException and rethrow as
   AuraHandledException with a friendly message ("That student is already enrolled on this cohort."
   for duplicate value errors), return the saved record with related names.

8) Tests: TestDataFactory (@isTest: createCourse, createCohort, createStudents with duplicate-rule
   bypass, buildEnrolments NOT inserted) plus EnrolmentTriggerTest (defaults, overbooking negative
   test, 200-record bulk, duplicate blocked, queueable via startTest/stopTest) and CohortServicesTest
   (controllers + both invocable services incl. error paths). Target 90% coverage, real assertions.

Checkpoint after deploy: enrolling a 4th student on a capacity-3 cohort must fail with the capacity
message, and a duplicate enrolment must fail on the unique key.
```

B.3 — Prompt: create all the Lightning Web Components

```
Same project; Apex from prompt B.2 is deployed. Create three LWCs under force-app/main/default/lwc,
each three files (html, js, js-meta.xml), apiVersion 62.0, isExposed true, targets lightning__AppPage,
lightning__HomePage, lightning__RecordPage. Then deploy.

1) cohortExplorer: lightning-combobox "Filter by course" fed by @wire CohortController.getActiveCourses
   (prepend an "All courses" option); lightning-datatable of @wire getOpenCohorts(courseId:
   '$selectedCourseId') with columns Cohort, Course (flattened Course__r.Name), Starts (date), Mode,
   Capacity, Seats left; single row selection dispatches CustomEvent 'cohortselected' (bubbles,
   composed) with the row; store the whole wire result and expose @api refresh() that calls
   refreshApex on it; friendly empty state.
```

```
2) enrolmentPanel: @api cohort; when set, show cohort name + a lightning-badge "<n> seat(s)
   remaining"; lightning-record-picker for Contact; "Enrol student" button disabled until a student
   is chosen; imperative call to EnrolmentController.createEnrolment with status 'Confirmed';
   on success ShowToastEvent, dispatch 'enrolled' (bubbles, composed), NavigationMixin to the new
   record; on error show error.body.message in a toast - never a stack trace; spinner while working.

3) enrolmentConsole: wrapper with a responsive SLDS grid - cohortExplorer (2/3 width) listening for
   oncohortselected to set selectedCohort, enrolmentPanel (1/3) receiving cohort={selectedCohort}
   and onenrolled calling the explorer's refresh().

Do not restyle beyond SLDS classes. After deploy, tell me how to add enrolmentConsole to a Lightning
App Page named "Enrolment Console" and activate it.
```

B.4 — Prompt: create all the flows

```
Same project. Create these three flows as flow-meta.xml files (API 62.0, status Active) and deploy.
If any element is easier done in Flow Builder, generate the closest valid metadata and tell me what
to verify in the Builder afterwards.

1) Enrolment_Confirmed_Actions - record-triggered on Enrolment__c, create-or-update, entry
   Status__c = 'Confirmed', only when updated to meet the condition, after-save. Elements:
   Get_Cohort (first Cohort__c where Id = $Record.Cohort__c); Send Email action to
   $Record.Student__r.Email, subject "Your place is confirmed - " + $Record.Cohort__r.Cohort_Label__c;
   Create a Task (Subject "Send welcome pack to " + $Record.Student__r.Name, WhatId = $Record.Id,
   OwnerId = $Record.OwnerId, ActivityDate = today, Priority High); Decision Is_Cohort_Now_Full
   (Get_Cohort.Seats_Remaining__c <= 0) -> Update the cohort Status__c = 'Full'.

2) Quick_Enrol_Student - screen flow: Screen 1 picklist choice set of active Course__c; Get open
   cohorts of the chosen course sorted by Start_Date__c; Screen 2 radio choice of cohort showing
   Cohort_Label__c with seats in help text; Screen 3 Contact lookup; Apex action Find Available
   Cohorts re-checking seats; Decision; Create Enrolment__c (Status 'Confirmed'); success screen
   showing the enrolment Name; "just filled up" screen otherwise; every Get/Create has a Fault
   connector to an error screen showing $Flow.FaultMessage.

3) Cohort_Start_Reminder - schedule-triggered daily 07:00 on Enrolment__c where Status__c =
   'Confirmed' and Cohort__r.Start_Date__c = today + 3 (formula); send a reminder email and create
   a Task.

After deploy, confirm all three show State = Active, and list any element I should visually verify
in Flow Builder.
```

B.5 — Prompt: create everything else (permission set, page, sample data, deploy)

```
Same project. Finish the delivery layer:

1) Permission set Enrolment_Console_User (label "Enrolment Console User", description referencing
   NFR-05: access by permission set, never by profile edits): object permissions Read/Create/Edit
   on Course__c, Cohort__c, Enrolment__c and Read on Contact; field permissions on all custom fields
   (formulas and roll-ups read-only); Apex class access CohortController, EnrolmentController,
   CohortAvailabilityService, EnrolmentStatusService; flow access Quick_Enrol_Student; tab
   visibility for Courses, Cohorts, Enrolments. Deploy, then assign it to me:
   sf org assign permset -n Enrolment_Console_User

2) Lightning App Page: FlexiPage named Enrolment_Console ("Enrolment Console"), two-region layout,
   with the enrolmentConsole LWC - deploy it and tell me how to activate it and add it to navigation.

3) Sample data script scripts/apex/seed.apex: two courses (Introductory fee 500, Advanced fee 1500),
   cohort CO-style capacity 3 starting +14 days and capacity 10 starting +30 days (Status Open),
   three Contacts with @oxfordable.test emails - idempotent (do not duplicate on re-run). Run it
   with sf apex run.

4) Verify the whole build: run all local tests with coverage and show the summary; then do a
   validate-only deploy of force-app to confirm everything is deployable:
```

```
   sf project deploy start --source-dir force-app --dry-run -l RunLocalTests

Finally, print a checklist of the manual steps that metadata cannot do: activate the app page,
check email deliverability is All Email, and run the Lab 2-4 checkpoints.
```

ONE-SHOT MASTER PROMPT (REBUILD EVERYTHING)

For a fresh org, you can paste prompts B.1–B.5 into the assistant **one at a time, in order, deploying and checkpointing between each**. Resist the temptation to merge them into a single mega-prompt: layer-by-layer builds surface errors where they happen, exactly like the labs do — and reviewing five small diffs teaches you more than approving one huge one. The Extension Pack features (invoicing, payments, certificates, portal, error logging) have their own build path: follow the Complete Build Guide, whose Code Pack items are the source of truth for that code.